

QuickAI — Project Documentation & Report

1. Project Overview

QuickAI is a full-stack AI-powered SaaS web application that allows users to generate content, process images, and review resumes using AI. It features a freemium model with plan-based access control.

- **Frontend:** React 19 + Vite + Tailwind CSS
- **Backend:** Node.js + Express 5
- **Auth:** Clerk (JWT-based)
- **Database:** Neon (PostgreSQL serverless)
- **Storage:** Cloudinary
- **AI:** Google Gemini 2.5 Flash (via OpenAI-compatible API) + ClipDrop

2. Project Structure

```
QuickAI/
├── client/                # React Frontend (Vite)
│   ├── src/
│   │   ├── components/   # Reusable UI components
│   │   ├── pages/        # Route-level page components
│   │   ├── assets/       # Static assets & asset exports
│   │   ├── App.jsx       # Router setup
│   │   └── main.jsx      # Entry point
└── server/               # Node.js Backend (Express)
    ├── configs/          # DB, Cloudinary, Multer setup
    ├── controllers/      # Business logic (AI + User)
    ├── middlewares/      # Auth middleware (Clerk)
    ├── routes/           # API route definitions
    └── server.js         # Express app entry point
```

3. Tech Stack

Layer	Technology
Frontend	React 19, Vite, Tailwind CSS v4
Routing	React Router DOM v7
Auth	Clerk (React + Express SDK)
HTTP Client	Axios
UI Extras	Lucide React, React Hot Toast, React Markdown
Backend	Node.js, Express 5
Database	Neon (PostgreSQL via @neondatabase/serverless)
File Upload	Multer

Cloud Storage	Cloudinary
AI (Text)	Google Gemini 2.5 Flash (OpenAI-compatible endpoint)
AI (Image)	ClipDrop API (text-to-image)
PDF Parsing	pdf-parse-fork
Deployment	Vercel (both client & server)

4. Features

Free Plan (up to 10 uses)

Feature	Route
Write Article	POST /api/ai/generate-article
Generate Blog Titles	POST /api/ai/generate-blog-title

Premium Plan Only

Feature	Route
Generate Images	POST /api/ai/generate-image
Remove Image Background	POST /api/ai/remove-image-background
Remove Object from Image	POST /api/ai/remove-image-object
Review Resume (PDF)	POST /api/ai/resume-review

User Features

Feature	Route
Get My Creations	GET /api/user/get-user-creations
Get Community Posts	GET /api/user/get-published-creations
Like/Unlike Creation	POST /api/user/toggle-like-creation

5. Frontend Pages & Components

Pages

Page	Path	Description
Home	/	Landing page with Hero, AiTools, Testimonials, Plans, Footer
Dashboard	/ai	Shows user's total creations and active plan

Write Article	/ai/write-article	AI article generator
Blog Titles	/ai/blog-titles	AI blog title generator
Generate Images	/ai/generate-images	Text-to-image (premium)
Remove Background	/ai/remove-background	BG removal (premium)
Remove Object	/ai/remove-object	Object removal (premium)
Review Resume	/ai/review-resume	PDF resume review (premium)
Community	/ai/community	Browse published AI creations

Components

Component	Purpose
Navbar	Top navigation bar
Hero	Landing page hero section
AiTools	AI features showcase grid
Plan	Pricing/plan selection UI
Testimonial	User testimonials carousel
Footer	Site footer
Sidebar	Dashboard side navigation
CreationItem	Single creation card in dashboard

6. API Documentation

Authentication

All protected routes require a Bearer token in the Authorization header:

```
Authorization: Bearer <clerk_jwt_token>
```

AI Endpoints — POST /api/ai/

Generate Article

- **Body:** { prompt: string, length: number }
- **Auth:** Required | **Plan:** Free (up to 10) / Premium
- **Response:** { success: true, content: string }

Generate Blog Title

- **Body:** { prompt: string }
- **Auth:** Required | **Plan:** Free (up to 10) / Premium
- **Response:** { success: true, content: string }

Generate Image

- **Body:** { prompt: string, publish: boolean }
- **Auth:** Required | **Plan:** Premium only
- **Response:** { success: true, content: imageUrl }

Remove Image Background

- **Body:** multipart/form-data with image file
- **Auth:** Required | **Plan:** Premium only
- **Response:** { success: true, content: imageUrl }

Remove Image Object

- **Body:** multipart/form-data with image file + { object: string }
- **Auth:** Required | **Plan:** Premium only
- **Response:** { success: true, content: imageUrl }

Review Resume

- **Body:** multipart/form-data with resume PDF file (max 5MB)
- **Auth:** Required | **Plan:** Premium only
- **Response:** { success: true, content: markdownString }

User Endpoints — /api/user/

GET /get-user-creations

Returns all creations for the logged-in user, sorted by date.

GET /get-published-creations

Returns all publicly published creations.

POST /toggle-like-creation

- **Body:** { id: number }
- Toggles like/unlike on a creation.

7. Database Schema

```
creations (  
  id          SERIAL PRIMARY KEY,  
  user_id     TEXT NOT NULL,  
  prompt      TEXT,  
  content     TEXT,  
  type        TEXT,          -- 'article' | 'blog-title' | 'image' | 'resume-review'  
  publish     BOOLEAN DEFAULT false,  
  likes       TEXT[],        -- array of user IDs who liked  
  created_at  TIMESTAMP DEFAULT NOW()  
)
```

8. Environment Variables

Server (server/.env)

```
PORT=3000
DATABASE_URL=<neon_postgres_connection_string>
GEMINI_API_KEY=<google_gemini_api_key>
CLIPDROP_API_KEY=<clipdrop_api_key>
CLOUDINARY_CLOUD_NAME=<cloudinary_cloud_name>
CLOUDINARY_API_KEY=<cloudinary_api_key>
CLOUDINARY_API_SECRET=<cloudinary_api_secret>
CLERK_SECRET_KEY=<clerk_secret_key>
```

Client (client/.env)

```
VITE_BASE_URL=<backend_api_url>
VITE_CLERK_PUBLISHABLE_KEY=<clerk_publishable_key>
```

9. Setup & Installation

Prerequisites

- Node.js >= 18
- Neon PostgreSQL database
- Clerk account
- Cloudinary account
- Google Gemini API key
- ClipDrop API key

Steps

```
# 1. Clone the repository
git clone <repo-url>
cd QuickAI

# 2. Install server dependencies
cd server && npm install

# 3. Install client dependencies
cd ../client && npm install

# 4. Configure environment variables
# Fill in server/.env and client/.env with your keys

# 5. Run development servers
# Terminal 1 - Backend
cd server && npm run server

# Terminal 2 - Frontend
cd client && npm run dev
```

10. Key Issues Found

The full code review found 30+ issues. Key areas to address:

#	Issue	Severity
1	.env files may be committed to git	Critical
2	toggleLikeCreation returns undefined creations variable	High
3	resumeReview references undefined pdfParse variable	High
4	No HTTP status codes — all errors return 200 with success: false	Medium
5	Auth middleware makes unnecessary Clerk API call for every premium request	Medium
6	No input validation on request body fields	Medium
7	Dead/commented-out code in resumeReview controller	Low
8	Wrong model name used: gemini-3-flash-preview (does not exist)	High
9	Multer stores files with no size limit configured	Medium
10	CORS is open (app.use(cors())) with no origin restriction	Medium

Open the **Code Issues Panel** in your IDE for the full list with suggested fixes.

11. Deployment

Both client and server are configured for Vercel deployment via `vercel.json` files.

- **Frontend:** Static Vite build deployed to Vercel
- **Backend:** Serverless Express deployed to Vercel

12. Architecture Diagram



